

LightCCI compare_N pairwise NMF 修改计划

中文 Codex Plan 讲义版 - 给你快速确认 plan 里到底要 Codex 做什么

一句话目标

把 compare_N_* 从 “全时间 temporal joint NMF ” 改成 “ 每个 time-pair 单独做 NMF ”：spot 用 shared-core directed NMF；gene / seurat_k40 / seurat_k150 用普通 pairwise joint NMF。

项目	内容
网络结构	LightCCI / light_cci
涉及 PIJ 方法	所有包含 N 的 compare 方法
核心原则	不使用全部时间点共同学习 NMF；只使用当前 source-target time-pair
spot 处理	不投影到 domain anchor；用 shared-core directed NMF 处理不同节点数
domain/gene 处理	节点数固定，使用普通 pairwise joint NMF

1. 为什么要改

当前 compare_N_* 的 N 特征来自全时间 temporal joint NMF。它会把 11.5、12.5、13.5、14.5 一起放进同一个 NMF 过程里，学习共享 H。这样虽然 component 对齐强，但对因果/跨时间 PIJ 任务会产生信息泄漏风险。

另外，spot 层每个时间点的节点数不一样，导致原始 adjacency 列数不一致，当前 NMF 入口会直接报错。

问题定位

当前核心问题在 `mignet_ce/pij/compare/features.py::_build_nmf_side()`：它检查所有时间点 adjacency 的列数必须一致，然后调用 `engine.temporal_joint_nmf()`。这对 spot 层不成立，也会引入全时间特征泄漏。

2. 最终认可方案

层级	新 NMF 方式	是否需要列数一致	是否使用其他时间点
spot	pairwise shared-core directed NMF	不需要	否，只用当前 pair
gene	ordinary pairwise joint NMF	需要，但 gene 固定 2000	否，只用当前 pair
seurat_k40	ordinary pairwise joint NMF	需要，但固定 40	否，只用当前 pair
seurat_k150	ordinary pairwise joint NMF	需要，但固定 150	否，只用当前 pair

3. 数学定义

3.1 固定节点层级：普通 pairwise joint NMF

对每个 source-target time-pair 单独做：

$$A_s \approx W_s H_{\{s,t\}}$$

$$A_t \approx W_t H_{\{s,t\}}$$

$$W_s: n \times r$$

$$W_t: n \times r$$

$$H_{\{s,t\}}: r \times n$$

$$N_s = W_s$$

$$N_t = W_t$$

这里的 H 只属于当前 time-pair，不会被其他时间点共享。

3.2 spot 层：pairwise shared-core directed NMF

spot 的 source 和 target 节点数可能不同，因此不用共享 H，而是共享中间的 component interaction core B：

$$A_s \approx U_s B_{\{s,t\}} V_s^T$$

$$A_t \approx U_t B_{\{s,t\}} V_t^T$$

$$U_s: n_s \times r \quad V_s: n_s \times r$$

$$U_t: n_t \times r \quad V_t: n_t \times r$$

$$B_{\{s,t\}}: r \times r$$

$$N_s = \text{concat}(U_s, V_s)$$

$$N_t = \text{concat}(U_t, V_t)$$

为什么不用 domain anchor

spot 的 N 特征不投影到 seurat_k40 / seurat_k150，因为那会把上层 domain 信息提前注入 lower 层，污染 spot -> domain 的层级比较。shared-core NMF 不需要 domain anchor，也不要求 spot 数量一致。

4. Codex 需要改哪些代码

文件	修改任务
mignet_ce/metrics.py	新增 pairwise_joint_nmf() 和 pairwise_shared_core_directed_nmf()
mignet_ce/pij/compare/features.py	把 _build_nmf_base() 改成 pairwise N 特征构建；支持 spot shared-core
mignet_ce/pij/compare/common.py	计算每个 pair kernel 时优先使用 feature_set.pairwise_*_features
mignet_ce/pij/base.py	可复用 PairFeatures；必要时扩展 MethodResult / CompareFeatureSet 类型
scripts/profile_lightcci_compare_matrix.py	更新 profile：N 类不再报告 spot 列数不一致为失败，而报告 shared-core 可行性

5. 数据结构改造

当前 CompareFeatureSet 只有按时间点保存的 lower_features / upper_features。N 特征改成 pairwise 后，需要新增 pairwise feature 字段：

```
pairwise_lower_features[(t0, t1)] = (lower_source_features, lower_target_features)
pairwise_upper_features[(t0, t1)] = (upper_source_features, upper_target_features)
```

common.py 中不能再无条件使用 feature_lists[pair[0]] 和 feature_lists[pair[1]]。当 pairwise feature 存在时，必须优先使用 pairwise feature。

6. 具体实现任务块

- 任务 1：在 metrics.py 中实现 ordinary pairwise joint NMF。输入两个列数一致的矩阵，返回 W_source、W_target、H。
- 任务 2：在 metrics.py 中实现 spot 用的 shared-core directed NMF。输入两个方阵，允许节点数不同，返回 U/V/B。
- 任务 3：重写 features.py::_build_nmf_base()。对每个 time-pair 分别构建 N 特征；spot 分派 shared-core，其他层级分派 ordinary pairwise joint NMF。
- 任务 4：扩展 CompareFeatureSet，使其能保存 pairwise_lower_features 和 pairwise_upper_features。
- 任务 5：修改 common.py，计算每个 pair 的 PIJ kernel 时优先读取 pairwise feature。
- 任务 6：修改 NMF artifact 导出。普通 pairwise 导出 H/W，spot shared-core 导出 B/U/V 和 concat 后的 feature。
- 任务 7：更新 profile 脚本和 metadata，明确记录 uses_only_pair_timepoints=true、uses_domain_anchor=false。

7. 标准化规则

pairwise N 特征的 component space 是当前 pair 专属的，因此不要再做全时间统一 scaler。建议对每个 pair、每个 side 单独 fit z-score：

```
fit on concat([N_source, N_target])
transform N_source and N_target
metadata: standardization = pairwise_side_specific_zscore
```

8. 受影响的方法

所有包含 N 的 compare 方法都要走新的 pairwise NMF：

compare_N_cos	compare_N_kl	compare_N_sot
compare_E_N_cos	compare_E_N_kl	compare_E_N_sot
compare_N_L_cos	compare_N_L_kl	compare_N_L_sot
compare_N_Sr_cos	compare_N_Sr_kl	compare_N_Sr_sot

不包含 N 的方法不改变核心数学定义：

compare_E_*	compare_L_*	compare_Sr_*
compare_E_L_*	compare_E_Sr_*	compare_L_Sr_*
compare_main_lap_sr_spatial_sot		

9. 用户命令不变

这个修改应该让原来的命令继续可用，不要求用户新增参数。N 类方法应该能够带完整 6 个层级对运行：

```
--level-pairs gene:spot gene:seurat_k40 gene:seurat_k150 spot:seurat_k40 spot:seurat_k150
seurat_k40:seurat_k150
```

结果预期

heart、4 个时间点、6 个 level-pair 时，每个 N 类方法应输出 $6 \times 6 = 36$ 行 metrics，不再因为 spot 列数不一致而跳过。

10. 测试计划

10.1 单元测试

- pairwise_joint_nmf：两个列数一致矩阵，检查 W_source / W_target / H shape。
- pairwise_joint_nmf：列数不一致时抛出清晰错误。
- pairwise_shared_core_directed_nmf：8x8 和 5x5 输入，检查 U/V/B shape，feature 维度为 2r。
- build_compare_feature_set：包含 N 且 layer=spot 时返回 pairwise feature，不再报 column count error。
- ComparePijMethodBase.run：确认 kernel 使用 pairwise features 而不是 timewise fallback。

10.2 集成测试

```
python -u scripts/run_mignet_vertical.py --data-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory" --output-root "/tmp/lightcci_pairwise_nmf_smoke" --network-method light_cci --pij-method compare_N_cos --organs heart --time-points 11.5 12.5 --level-pairs gene:spot spot:seurat_k40 seurat_k40:seurat_k150 --development-feature-root "/home/jovyan/work/2026 Causality/output/developmental_features" --export-pij --export-pij-topk 0 --progress --max-workers 16
```

预期：run_summary.csv 全部 status=written；metrics.csv 有 3 行。完整 4 时间点、6 层级对时，metrics.csv 有 36 行。

11. 验收标准

- compare_N_* 可以带 spot 层运行。
- compare_E_N_* / compare_N_L_* / compare_N_Sr_* 可以带 spot 层运行。
- 不再出现 “compare N feature requires temporal joint NMF inputs with identical column counts”。
- gene:spot、spot:seurat_k40、spot:seurat_k150 都能输出 metrics。
- metadata 明确记录
pairwise_nmf、spot_shared_core_directed_nmf、uses_only_pair_timepoints=true、uses_domain_anchor=false。

12. 明确不要做

- 不要再把全时间 temporal_joint_nmf 作为 compare_N 的默认特征。
- 不要把 spot 投影到 seurat_k40 / seurat_k150 domain anchor。
- 不要要求 spot 在不同时间点有相同节点数。
- 不要改变 E、L、Sr 的核心数学定义。
- 不要改变用户现有命令格式。

13. 推荐 Codex 提交说明

Commit message:

Refactor LightCCI compare N features to pairwise NMF

Summary:

Replace all-time temporal joint NMF in LightCCI compare_N features with pairwise NMF. Fixed-size layers use ordinary pairwise joint NMF, while spot layers use pairwise shared-core directed NMF to support variable spot counts without domain anchors. This avoids temporal leakage across unrelated time points and enables N-containing compare methods to run on spot-related level pairs.